ask the question, “Here’s a new list, give me the hundred existing lists closest to it.” That’s the entire core operation. Everything else is convenience features on top.

Why a regular database struggles with this

You might think, fine, just put the numbers in a Postgres table with a thousand columns and write a query. Let me explain why that falls apart.

The problem is the cost of comparison. To find the vectors closest to your query vector, you must compare your vector against every stored vector. With a few thousand vectors, that’s fine. With ten million, the process is very slow. With a billion, it becomes basically impossible to do in real time.

Vector databases solve this by building index structures that let you skip most of the comparisons. The most popular approach is something called HNSW, which stands for Hierarchical Navigable Small World. The details don’t matter much for application developers, but you can find approximately the closest matches in logarithmic time instead of linear time. Searching a billion vectors becomes possible in milliseconds.

You don’t always get the exact closest matches. You get something very close to them, almost always, with much better performance. For nearly all real applications, this trade-off is fine. Nobody cares whether they got the actual top ten most similar support tickets or the top ten most similar tickets out of the actual top fifteen. The difference is invisible to the user.

The other thing vector databases do that’s hard to replicate in regular databases is metadata filtering combined with vector search. You can ask, “Find me documents similar to this query, but only ones from the last thirty days, only in English, only tagged with ‘invoice’.” Doing this efficiently while also doing approximate nearest neighbour search is its own engineering puzzle, and the better vector databases handle it well.

The open source landscape, briefly

There are a lot of options here, and the landscape moves fast. But here’s the rough lay of the land as it stands.

Qdrant is written in Rust, very fast, has a clean API, and supports rich filtering. It’s become one of my personal favourites for projects where I need to self-host. The community is active and the docs are good.

Milvus is one of the older names, originally from Zilliz. It’s designed for very large scale, supports multiple index types, and integrates well with cloud-native infrastructure. If you’re going to have hundreds of millions of vectors, Milvus is worth a serious look.

Weaviate has a slightly different philosophy. It’s built around a schema with semantic understanding, can do hybrid search combining vector and keyword search out of the box, and has some interesting modules for working with images and other media. It’s a bit heavier to run but powerful.

Chroma is the easy-on-ramp option. Lightweight, easy to embed in Python applications, popular for prototypes and small projects. Not the fastest at scale, but the friction to get started is the lowest of any option here.

pgvector is the interesting middle path. It’s an extension for Postgres that adds vector search capabilities. Performance isn’t as good as the purpose-built options at very large scale, but if you’re already using Postgres, the operational simplicity of not running a separate database is huge. For many real-world applications, pgvector is more than enough.

There are others. FAISS from Meta, technically a library more than a database. LanceDB for embedded use cases. Vespa from Yahoo, which is very capable for hybrid search. The list keeps growing.

I’d suggest starting with either Chroma or pgvector for prototyping, and moving to Qdrant or Milvus if you find yourself needing more performance or features.

Let’s use a vector database

Theory only gets you so far. Let me show you what working with a vector database looks like, using Qdrant as an example.

First, get it running. The easiest way is Docker.

```
docker run -p 6333:6333 qdrant/qdrant
```

That’s it. Qdrant is now running on port 6333, ready to accept connections.

Now in your Python code, install the client and an embedding model.

```
pip install qdrant-client sentence-transformers
```

Here’s a tiny script that creates a collection, adds some documents, and searches them.

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
from sentence_transformers import SentenceTransformer

client = QdrantClient(host="localhost", port=6333)
model = SentenceTransformer("all-MiniLM-L6-v2")

client.create_collection(
    collection_name="docs",
    vectors_config=VectorParams(size=384, distance=Distance.COSINE),
)

documents = [
    "The cat sat on the mat.",
    "A dog ran through the park.",
    "Linux is a popular operating system.",
    "Python is widely used for data science.",
    "Cats and dogs are common household pets.",
]
```